

ORF307_HW2_soln

1 ORF 307 HW2 Soln

Q1 Least angle property and orthogonality principle of least squares

Suppose the $m \times n$ matrix A has linearly independent columns, and b is an m -vector. Let x^* denote the least squares approximate solution of $Ax = b$, i.e., $A^T Ax^* = A^T b$.

- (a) Show that for any n -vector x , $(Ax)^T b = (Ax)^T (Ax^*)$, i.e., the inner product of Ax and b is the same as the inner product of Ax and Ax^* .

Hint. Use $(Ax)^T b = x^T (A^T b)$.

- (b) Show that when Ax^* and b are both nonzero, we have

$$\frac{(Ax^*)^T b}{\|Ax^*\|_2 \|b\|_2} = \frac{\|Ax^*\|_2}{\|b\|_2}.$$

The left-hand side is the cosine of the angle between Ax^* and b .

Hint. Apply part (a) with $x = x^*$.

- (c) *Least angle property of least squares.* Show that $x = x^*$ also minimizes the angle between Ax and b . (You can assume that Ax and b are nonzero.)

Remark. For any positive scalar α , $x = \alpha x^*$ also minimizes the angle between Ax and b .

- (d) *Orthogonality principle.* The optimal residual r is defined as $r = Ax^* - b$. Show that r is orthogonal to the columns of A , and therefore, it is orthogonal to any linear combination of the columns of A . In other words, for any n -vector z , we have $(Az)^T r = 0$.

Remark. This is one of the most important properties of least squares solutions: Ax^* is the linear combination of the columns of A that is closest to b and, it is orthogonal to the residual vector $r = Ax^* - b$. In other words, Ax^* is the “projection” of b on the space of linear combinations of the columns of A .

Solution

- (a)

$$(Ax)^T b = x^T A^T b = x^T (A^T b) = x^T (A^T A)x^* = (x^T A^T)(Ax^*) = (Ax)^T (Ax^*),$$

where in the third equation we use the *normal equations* $A^T b = (A^T A)x^*$.

- (b) Apply part (a) with $x = x^*$, we have $(Ax^*)^T b = (Ax^*)^T (Ax^*) = \|Ax^*\|_2^2$. Divide both sides by $\|Ax^*\|_2 \|b\|_2$, we get

$$\frac{(Ax^*)^T b}{\|Ax^*\|_2 \|b\|_2} = \frac{\|Ax^*\|_2}{\|b\|_2}.$$

- (c) Let θ be the angle between Ax and b . Then x^* minimizes the angle between Ax and $b \Leftrightarrow x^*$ maximizes $\cos \theta$ among all x . We know $\cos \theta$ between Ax and b is defined as

$$\frac{(Ax)^T b}{\|Ax\|_2 \|b\|_2}.$$

From (b) we know

$$\frac{(Ax^*)^T b}{\|Ax^*\|_2 \|b\|_2} = \frac{\|Ax^*\|_2}{\|b\|_2}.$$

Therefore, we only need to show that

$$\frac{\|Ax^*\|_2}{\|b\|_2} \geq \frac{(Ax)^T b}{\|Ax\|_2 \|b\|_2}$$

for any x . From Cauchy–Schwarz inequality, we have

$$\begin{aligned} & \|Ax^*\|_2 \|Ax\|_2 \geq (Ax)^T (Ax^*) \\ \Leftrightarrow & \|Ax^*\|_2 \|Ax\|_2 \geq x^T (A^T Ax^*) \\ \Leftrightarrow & \|Ax^*\|_2 \|Ax\|_2 \geq x^T A^T b \quad \text{normal equations} \\ \Leftrightarrow & \|Ax^*\|_2 \|Ax\|_2 \geq (Ax)^T b \\ \Leftrightarrow & \frac{\|Ax^*\|_2}{\|b\|_2} \geq \frac{(Ax)^T b}{\|Ax\|_2 \|b\|_2}. \end{aligned}$$

Equation holds if and only if $x = \alpha x^*$.

- (d)

$$(Az)^T r = (Az)^T (Ax^* - b) = z^T A^T (Ax^* - b) = z^T (A^T Ax^* - A^T b) = Z^T 0 = 0,$$

where in the third equation we use the *normal equations* $A^T b = (A^T A)x^*$.

Q2 Matrix inversion lemma

- (a) It is sometimes very cheap to solve subsequent linear systems when the coefficient matrix gets slightly modified. Suppose we have a $n \times n$ matrix M . Given n -vectors u and v , prove that, if $v^T M^{-1} u \neq -1$, the following update, called *matrix inversion lemma* holds:

$$(M + uv^T)^{-1} = M^{-1} - \frac{1}{1 + v^T M^{-1} u} M^{-1} uv^T M^{-1}.$$

Hint. Directly apply the matrix inverse definition, i.e., $(M + uv^T)^{-1}(M + uv^T) = I$.

Remark. The assumption $v^T M^{-1} u \neq -1$ can be shown to correspond to $M + uv^T$ being invertible.

- (b) Suppose now that M is symmetric positive definite and that you have already computed a Cholesky decomposition LL^T (therefore, we have a convenient way to solve $Mx = b$), but that we want to solve the linear system

$$(M + uv^T)x = b.$$

Explain how we can use the recursion in point (a) to do so by solving a few prefactored linear systems.

Hint. Remember, we never compute inverses explicitly! Every time you have $M^{-1}q$ for some vector n -vector q , it means solving the linear system $Mz = q$ with n -vector z .

- (c) Work out the total flop count for this method keeping only the dominant terms and compare it to factoring matrix $M + uv^T$ from scratch.

Solution

- (a) We only need to prove $\left(M^{-1} - \frac{1}{1+v^T M^{-1}u} M^{-1}uv^T M^{-1}\right)(M + uv^T) = I$. We have

$$\begin{aligned} & \left(M^{-1} - \frac{1}{1+v^T M^{-1}u} M^{-1}uv^T M^{-1}\right)(M + uv^T) \\ &= I - \frac{1}{1+v^T M^{-1}u} M^{-1}uv^T + M^{-1}uv^T - \frac{1}{1+v^T M^{-1}u} M^{-1}uv^T M^{-1}uv^T \\ &= I - \frac{1}{1+v^T M^{-1}u} M^{-1}uv^T + M^{-1}uv^T - \frac{1}{1+v^T M^{-1}u} M^{-1}u(v^T M^{-1}u)v^T \\ &= I - \frac{1}{1+v^T M^{-1}u} M^{-1}uv^T + M^{-1}uv^T - \frac{v^T M^{-1}u}{1+v^T M^{-1}u} M^{-1}uv^T \quad \text{Note: } v^T M^{-1}u \text{ is a scalar.} \\ &= I - \frac{1+v^T M^{-1}u}{1+v^T M^{-1}u} M^{-1}uv^T + M^{-1}uv^T = I. \end{aligned}$$

- (b) $(M + uv^T)x = b \Rightarrow x = (M + uv^T)^{-1}b$. Apply part (a) we have

$$x = M^{-1}b - \frac{1}{1+v^T M^{-1}u} M^{-1}uv^T M^{-1}b.$$

As we already have the Cholesky decomposition, we can solve the linear system within n^2 flops using the following steps.

- Solve the system $Mx_1 = u$ for x_1 in n^2 time.
- Solve the system $Mx_2 = b$ for x_2 in n^2 time.
- Compute $x = x_2 - \frac{1}{1+v^T x_1} x_1 v^T x_2$ in n^2 time.

- (c) In part (b) we have shown the flop count for this method, while factoring matrix $M + uv^T$ from scratch takes n^3 time.

Q3 Moore's law

```
[1]: # Import required packages for coding exercises
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from numpy.linalg import cholesky as llc
seed = 1
np.random.seed(seed) # We use np.random.seed for reproducible results
```

For convenience we have included the following function `lstsq` that solves the least squares problem as we have seen in class returning $\mathbf{x}$ and factorization $\mathbf{L}$.

```
[2]: def forward_substitution(L, b):
    n = L.shape[0]
    x = np.zeros(n)
    for i in range(n):
        x[i] = (b[i] - L[i,:i] @ x[:i])/L[i, i]
    return x

def backward_substitution(U, b):
    n = U.shape[0]
    x = np.zeros(n)
    for i in reversed(range(n)):
        x[i] = (b[i] - U[i,i+1:] @ x[i+1:])/U[i, i]
    return x

def lstsq(A, b):
    M = A.T.dot(A) # Form Gram matrix
    q = A.T.dot(b) # Form right hand side
    L = llh(M)      # Factor
    x = forward_substitution(L, q)
    x = backward_substitution(L.T, x)
    return x, L
```

The figure and table below show the number of transistors N in 13 microprocessors, and the year of their introduction.

```
[3]: # Data
t = np.array([1971, 1972, 1974, 1978, 1982, 1985, 1989, 1993, 1997, 1999, 2000,
↪2002, 2003])
N = np.array([2250, 2500, 5000, 29000, 120000, 275000, 1180000, 3100000,
↪7500000, 24000000, 42000000, \
                220000000, 410000000])

# Show data as dataframe
# NB. Jupyter makes a nice table out of a pandas dataframe
pd.DataFrame({'t': t,
              'N': N})
```

```
[3]:
```

	t	N
0	1971	2250
1	1972	2500
2	1974	5000
3	1978	29000
4	1982	120000
5	1985	275000
6	1989	1180000
7	1993	3100000
8	1997	7500000

```

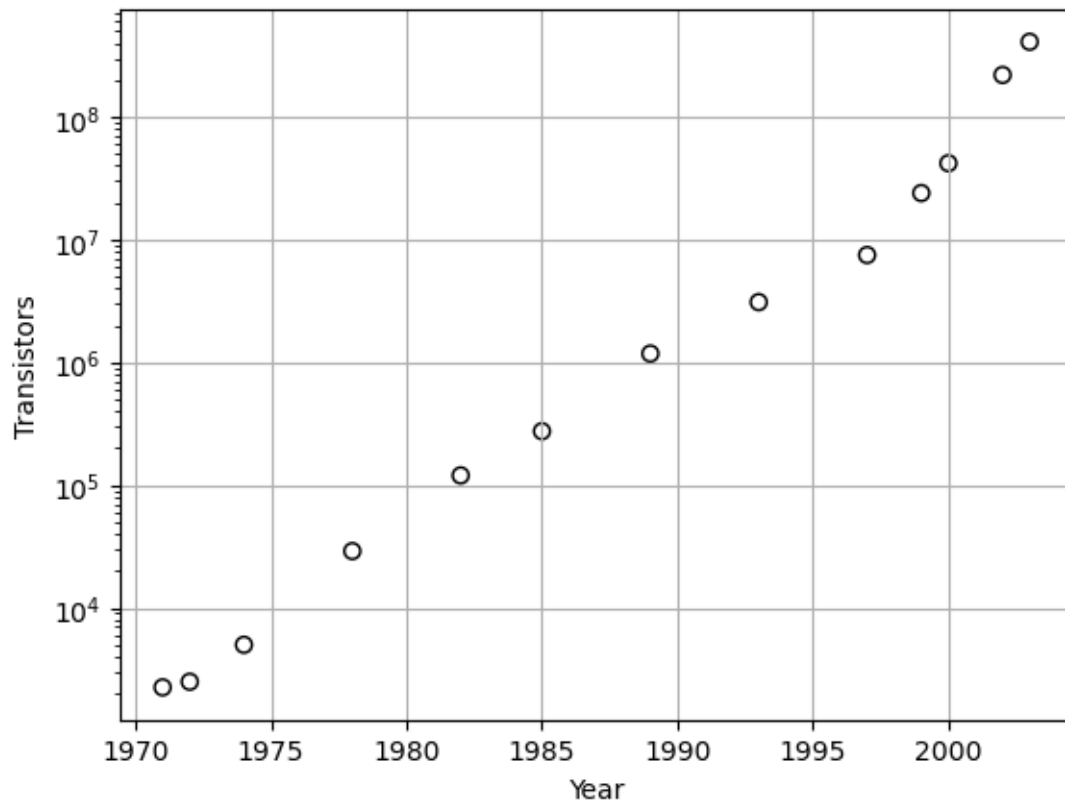
9   1999  24000000
10  2000  42000000
11  2002  220000000
12  2003  410000000

```

```

[4]: fig, ax = plt.subplots(1,1)
     ax.scatter(t, N, marker='o', color='k', facecolors='none')
     ax.set_xlabel(r'Year')
     ax.set_ylabel(r'Transistors')
     ax.set_yscale('log')
     ax.grid('on')

```



The plot gives the number of transistors on a logarithmic scale. Find the least squares straight-line fit of the data using the model

$$\log_{10} N \approx \theta_1 + \theta_2(t - 1970),$$

where t is the year and N is the number of transistors. Note that θ_1 is the model's prediction of the log of the number of transistors in 1970, and 10^{θ_2} gives the model's prediction of the fractional increase in number of transistors per year.

- (a) Find the coefficients θ_1 and θ_2 that minimize the mean square error on the data using the `lstsq` function above. Give the mean square error on the data. Plot the model you find along

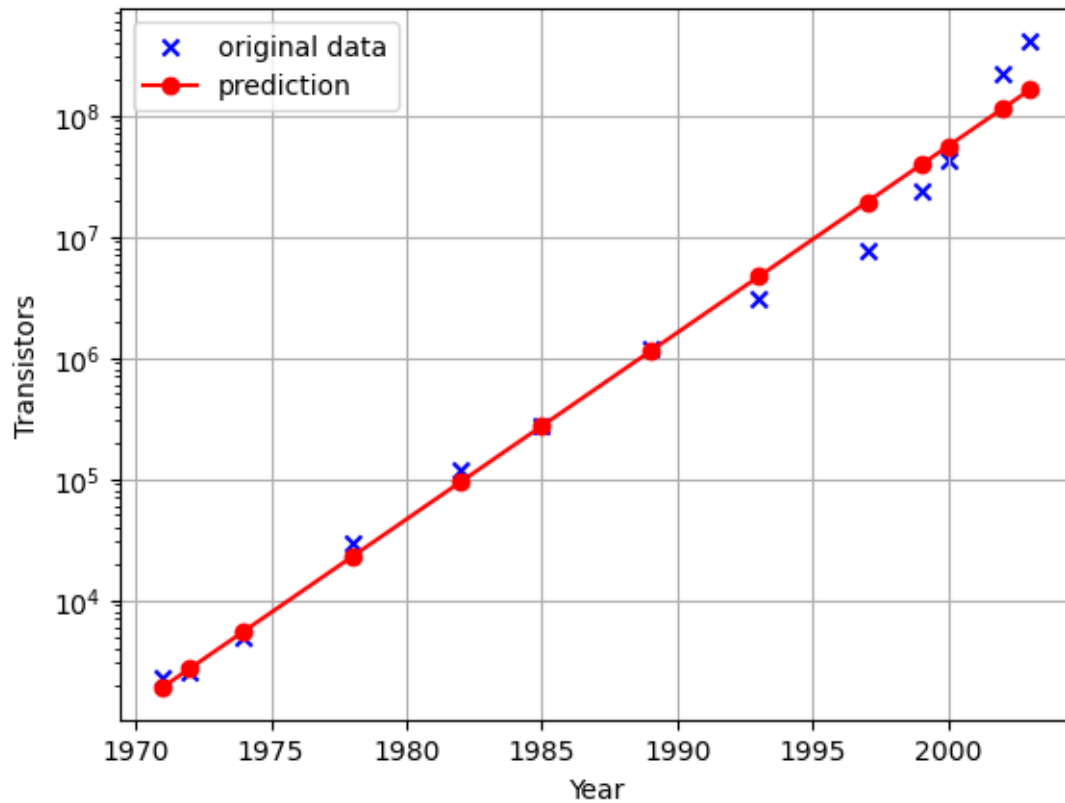
with the data points.

- (b) The first point-contact transistor was invented in 1947. Now let $N = 1$, where $\log_{10} N = 0$, and then find t . See if your model fits the historical data well.
- (c) Use your model to predict the number of transistors in a microprocessor introduced in 2015. Compare the prediction to the IBM Z13 microprocessor, released in 2015, which has around 4×10^9 transistors.
- (d) Compare your result with Moore's law, which states that the number of transistors per integrated circuit roughly doubles every one and a half to two years.

Solution

```
[5]: # (a)
A = np.ones((13, 2))
A[:, 1] = t - 1970
b = np.log10(N)
theta, _ = lstsq(A, b)
mse = np.linalg.norm(b - theta[0] - theta[1] * (t - 1970))**2/np.shape(t)[0]
predict = np.power(10, theta[0] + theta[1] * (t - 1970))
print('theta_1={}, theta_2={}, MSE={}'.format(theta[0], theta[1], mse))
fig, ax = plt.subplots(1,1)
ax.scatter(t, N, marker='x', color='blue', label='original data')
ax.plot(t, predict, marker='o', color='red', label='prediction')
ax.set_xlabel(r'Year')
ax.set_ylabel(r'Transistors')
ax.set_yscale('log')
ax.grid('on')
plt.legend(numpoints=1)
plt.show()
```

theta_1=3.125592633829349, theta_2=0.15401817984382243, MSE=0.04124014087055706



```
[6]: # (b)
print('Under our model, the first transistor was invented in', int(np.
    ↳round(-theta[0]/theta[1] + 1970)))
```

Under our model, the first transistor was invented in 1950

```
[7]: # (c)
print('Predicted number of transistors in 2015 is', np.power(10, theta[0] +
    ↳theta[1] * (2015 - 1970)))
```

Predicted number of transistors in 2015 is 11387036868.698635

(d) The number of transistors is doubled in time $\Delta t = t_2 - t_1$

$$\log_{10}(2N) - \log_{10}(N) = \theta_1 + \theta_2(t_2 - 1970) - \theta_1 - \theta_2(t_1 - 1970) \quad (1)$$

$$\log_{10}(2N/N) = \theta_2 \Delta t \quad (2)$$

$$\Delta t = \log_{10}(2)/\theta_2 \quad (3)$$

```
[8]: print('Under our model, the number of transistors per integrated circuit
    ↳doubles every {} years'\
        .format(np.log(2) / np.log(10) / theta[1]))
```

Under our model, the number of transistors per integrated circuit doubles every
1.9545094999124888 years

[]: